

Middleware Architectures 2

Lecture 8: Scheduling, Scalability and HPA

doc. Ing. Tomáš Vitvar, Ph.D.

tomas@vitvar.com • @TomasVitvar • <https://vitvar.com>



Czech Technical University in Prague

Faculty of Information Technologies • Software and Web Engineering • <https://vitvar.com/lectures>



Evropský sociální fond
Praha & EU: Investujeme do vaší budoucnosti

Modified: Sun May 03 2026, 20:38:11
Humla v1.0

Overview

- Kubernetes Scheduling
- Scalability and HPA

The Kubernetes Scheduler

- The **kube-scheduler** is a control-plane component responsible for Pod placement
 - *Watches for newly created Pods with no assigned node*
 - *Selects the best node for each unscheduled Pod and writes the binding*
 - *Does not start or run the Pod – done by the kubelet on the node*
- Scheduling pipeline runs in two phases
 - **Filtering** – *eliminates nodes that cannot run the Pod (hard constraints)*
 - **Scoring** – *ranks remaining nodes and picks the highest-score one*
- Scheduler is pluggable via the **Scheduling Framework**
 - *Extension points: **PreFilter**, **Filter**, **PostFilter**, **PreScore**, **Score**, **Reserve**, **Bind***
 - *Custom plugins can be compiled in or loaded as out-of-tree schedulers*
- Multiple scheduler profiles can coexist in a single cluster

Filtering – Hard Constraints

- A node passes filtering only when *all* predicates are satisfied
 - **Resource fit** – requested CPU/memory must fit into allocatable capacity
 - **Node selector** – Pod's `nodeSelector` labels must match node labels
 - **Node affinity** – `requiredDuringSchedulingIgnoredDuringExecution` rules
 - **Taints and tolerations** – Pod must tolerate all taints on the node
 - **Volume topology** – PVCs must be satisfiable on the candidate node
 - **Pod anti-affinity** – required rules must not be violated
- Example: scheduling a GPU workload

```
1 spec:
2   nodeSelector:
3     accelerator: nvidia-tesla-a100
4   tolerations:
5     - key: "gpu"
6       operator: "Exists"
7       effect: "NoSchedule"
8   containers:
9     - name: trainer
10      image: pytorch/pytorch:2.3
11      resources:
12        limits:
13          nvidia.com/gpu: "1"
```

- Pods that cannot be scheduled remain in **Pending** state

Scoring – Soft Preferences

- After filtering, each remaining node receives a score 0–100
 - *Final node score is a weighted sum across all enabled scoring plugins*
- Final score formula
 - $\text{finalScore}(\text{node}) = \sum \text{pluginScore}(\text{node}, p) \times \text{pluginWeight}(p)$
 - *The node with the highest **finalScore** is selected; ties are broken randomly*
- Key scoring plugins
 - ***LeastAllocated** – prefers nodes with most free CPU and memory*
 - ***MostAllocated** – bin-packing; fills nodes before using new ones*
 - ***NodeAffinity** – rewards nodes matching preferred affinity rules*
 - ***InterPodAffinity** – rewards co-location or spread based on soft rules*
 - ***ImageLocality** – prefers nodes that already have the container image*
 - ***TaintToleration** – penalises nodes with tolerated-but-present taints*

Kubernetes Lease-Based Leader Election

- Each controller that needs a leader creates a `coordination.k8s.io/Lease` object
 - The holder writes its identity and a **renewTime** timestamp periodically
 - If **renewTime** is stale by more than **leaseDuration**, a candidate can acquire the lease
 - Optimistic concurrency (resource version) prevents two candidates from acquiring simultaneously
- Parameters
 - **leaseDuration** – how long a lease is valid without renewal (default 15 s)
 - **renewDeadline** – leader must renew within this window (default 10 s)
 - **retryPeriod** – how often candidates retry acquisition (default 2 s)
- Illustrated state machine

```
1  
2  
3 Candidate A  
4 GET Lease → leaseDuration expired?  
5   YES → PUT Lease(holder=A, renewTime=now)  
6       → success (no conflict) → LEADER  
7       → 409 conflict (B won) → wait+retry  
8 As LEADER: every renewDeadline/2 renew lease  
9 On crash: lease expires → B or C takes over
```

- Inspecting the current leader:

Overview

- Kubernetes Scheduling
- Scalability and HPA

Horizontal vs. Vertical Scaling

- **Vertical scaling (scale-up)** – give the same instance more CPU/memory
 - *Simple to implement; no application changes required*
 - *Pod needs to be restarted (in-place resize is possible)*
- **Horizontal scaling (scale-out)** – add more replicas of the same component
 - *Near-linear throughput increase for stateless services*
 - *Fault tolerance improves with replica count*
 - *Requires applications to be **stateless** or use shared state (DB, cache)*
- **Comparison at a glance**
 - *Vertical: fast to apply, limited headroom, downtime risk, good for stateful databases*
 - *Horizontal: unlimited (in theory), zero-downtime, preferred for web/API tiers*
- Kubernetes supports both: **HPA** (horizontal) and **VPA** (vertical)

HPA Overview

- The **HorizontalPodAutoscaler** automatically adjusts the replica count of a workload
 - *Targets: **Deployment**, **ReplicaSet**, **StatefulSet**, or any resource implementing the scale subresource*
 - *Operates as a control loop running every 15 s (configurable)*
- Supported metric types
 - ***Resource metrics** – CPU/memory utilisation from **metrics-server***
 - ***Custom metrics** – application-specific metrics via **custom.metrics.k8s.io** API*
 - ***Object metrics** – a single metric on a specific Kubernetes object (e.g. Ingress RPS)*
- HPA does *not* start new nodes
 - *Cluster Autoscaler is needed when no node has capacity*

HPA Control Loop

- The HPA controller computes the desired replica count using the formula
 - `desiredReplicas = ceil(currentReplicas × (currentMetric / desiredMetric))`
 - *Example: 3 replicas, CPU at 80%, target 50% → $\text{ceil}(3 \times 80/50) = \text{ceil}(4.8) = 5$*
- Scale-down is stabilised to prevent flapping
 - *Default stabilisation window: **5 minutes** before scaling down*
 - *Scale-up is immediate (no default stabilisation)*
- Control flow
 - 1 Every 15 s:
 - 2 1. Fetch metric values **for** all Pods **in** the target
 - 3 2. Discard Pods **in** terminating or not-yet-ready state
 - 4 3. Compute desiredReplicas per metric; take the maximum
 - 5 4. **if** desiredReplicas < minReplicas → use minReplicas
 - 6 **if** desiredReplicas > maxReplicas → use maxReplicas
 - 7 5. If stabilisation window allows → apply scale
 - 8 6. Update HPA status (currentReplicas, desiredReplicas, conditions)
- Observe current state: `kubectl get hpa -w`
- Detailed events: `kubectl describe hpa <name>`

HPA Configuration Example

- HPA v2 API allows multiple metrics and fine-grained scaling behaviour

```
1  apiVersion: autoscaling/v2
2  kind: HorizontalPodAutoscaler
3  metadata:
4    name: api-server-hpa
5  spec:
6    scaleTargetRef:
7      apiVersion: apps/v1
8      kind: Deployment
9      name: api-server
10   minReplicas: 2
11   maxReplicas: 20
12   metrics:
13   - type: Resource
14     resource:
15       name: cpu
16       target:
17         type: Utilization
18         averageUtilization: 60
19   - type: Resource
20     resource:
21       name: memory
22       target:
23         type: AverageValue
24         averageValue: 512Mi
25   behavior:
26     scaleDown:
27       stabilizationWindowSeconds: 300
28       policies:
29       - type: Percent
30         value: 25
31         periodSeconds: 60
32     scaleUp:
33       policies:
34       - type: Pods
35         value: 4
36         periodSeconds: 30
```

Custom Metrics and KEDA

- Built-in CPU/memory metrics are insufficient for event-driven workloads
- **Custom Metrics API** – adapters expose application metrics to HPA
 - *Popular adapters: Prometheus Adapter, Datadog Cluster Agent*
 - *Example: scale on HTTP request rate from Prometheus*

```
1  # HPA using a custom Prometheus metric
2  metrics:
3  - type: Pods
4  pods:
5  metric:
6    name: http_requests_per_second
7  target:
8    type: AverageValue
9    averageValue: "500"
```

- **KEDA (Kubernetes Event-Driven Autoscaler)**
 - *Extends HPA with 60+ built-in scalers*
 - *Scales on RabbitMQ queue length, Kafka consumer lag, ...*